Day 8 (Evening Session)

Express and MySQL: Database Connectivity, Connection Pool and CRUD APIs

Lecture notes - Instructor: Abhishek Purohit - Date: 28 Oct 2026 - Notes for: Sumeet Sharma

#	Topic covered in this lecture
1	Architecture: client, API and database; insertion flow and fetch flow
2	CRUD and the product table in MySQL
3	Project setup: express and mysql2
4	Two ways to connect: a connection per request vs a connection pool
5	createPool with mysql2/promise and its configuration
6	Create (insert): POST /product, error handling, SQL injection and the ? placeholder
7	Read: GET /product (all) and why not to send everything; pagination
8	Read one: GET /product/:id with a 404 response
9	Delete: DELETE /product/:id
10	Assignments: update API, existence check, unique constraint error handling
11	The complete index.js
12	Revision: summary, definitions, differences, mistakes, interview Q&A, exercises, checklist

How to read these notes
Blue boxes hold the instructor's explanation in clear written English. Yellow boxes are additional explanations added by me; they were **not** said in the lecture. Red boxes list common mistakes. The complete code from your notes is included with explanation and the outputs shown in class. This lecture did not cover the DOM, events or forms, so those are not part of these notes.

1. The Architecture: Client, API and Database

Instructor's explanation
So far we created APIs with routes in Express. Now these APIs must **work with a database** (MySQL, MongoDB or any other): the Express application is connected to the underlying database, and then it can insert, fetch, update and delete data. On the left is the **client** (a web application, or an iOS or Android app). On the right is the **server**, which runs two things: the **API** (here an Express API; it could be built in Java or anything else) and the **database**.

Common mistake
The client **never talks to the database directly**. The client only calls the API and passes data in the request body. It is the job of the API to create the connection to the database and run the operation.

1.1 Insertion flow (for example registration or sign up)

1. The user fills a form in the front end and clicks submit; the front end calls the registration API with a **JSON request body**.
2. The API reads the request body and builds an **INSERT query**.
3. The API connects to the database and fires the insert query.
4. The database returns the result (success or failure) to the API.
5. The API sends a **response** (a success or failure JSON) back to the client.

1.2 Fetch flow

1. The client needs data (for example the products saved earlier); it sends a **request to the API**, not to the database.
2. The API connects to the database and fires a **SELECT query**.
3. The database runs it and returns the result (rows and columns) to the API.
4. The API converts the rows into **JSON (an array of JSON objects)** and sends it in the response.

Update and delete follow the same pattern with UPDATE and DELETE queries. The same principle holds whether the server is built with Java (Spring Boot with Hibernate), Python, .NET or Node.js: only the implementation differs.

Operation	SQL	HTTP method used in this lecture
Create	INSERT	POST
Read	SELECT	GET
Update	UPDATE	PUT (assignment)
Delete	DELETE	DELETE

These four operations are called **CRUD** (create, read, update, delete). In a real application the user never runs SQL directly: the front end calls an API and the API talks to the database.

2. The Database and the Table

In MySQL (version 8 in the demo; password 'root' on the instructor's machine) create a database and a table:

```
CREATE DATABASE productdb;
USE productdb;

CREATE TABLE product (
    id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    name VARCHAR(200),
    price FLOAT,
    description VARCHAR(300),
    qty INT
);

DESC product;
```

Explanation: id is the primary key and is **auto increment**, so you never insert the id; MySQL assigns it. name (200 characters), price (float), description (300 characters) and the quantity available (qty). DESC shows the structure: id is an integer, not null, primary key, auto_increment. (The instructor spoke of the database as 'product DB'; the code uses the name productdb. A missing comma in the first CREATE attempt gave a syntax error, which he fixed.)

3. Project Setup

1. Create a separate folder for the application (here CRUD APIs) and open it in VS Code.
2. Run **npm init -y** to create package.json; change **"type"** to **"module"** to use the latest ECMAScript import syntax.
3. Install both dependencies in one command: **npm i express mysql2** (or install them separately). Both become dependencies of the application.
4. Create index.js.

Instructor's explanation
We are not going to implement low-level communication between the Express server and the MySQL server ourselves. We use a third-party module: **mysql2**. It provides predefined functions to create connections, send queries to the database, and receive results (insert, fetch, update, delete). Underneath, it creates the connection to your database and runs your queries.

4. Two Ways to Connect: Per-Request Connection vs Connection Pool

There are two approaches (the idea is the same in every technology, not only Node.js):

	New connection for every request	Connection pool (use this)
Idea	Every request creates its own new database connection	A pool of **ready-made connections** is created in advance; requests borrow one and give it back
With 10 requests	10 new connections are created	10 of the pooled connections are occupied and then released
With lakhs of users	Lakhs of connections; the server is already struggling with traffic and would explode; time is wasted creating each connection	Limited number of connections (the developer decides); no repeated create and close
Where used	Only small systems (beginners do this)	**Every real-time/enterprise application** (Express, Java, Python, .NET, PHP)

Instructor's explanation
A connection pool is like a swimming pool holding ready-made connections. As developer you **decide the connection limit** (for example a maximum of 50). When a client request arrives, a ready-made connection is taken, the database operation is performed, and the response is sent; the connection is then **released back to the pool**, not closed. If all 50 are in use, further requests wait in a **queue**. You decide the **queue limit** too: how many requests may wait; beyond it requests are refused and the server sends a response such as 'resources not available' or 'high server traffic'. Whether a client should wait in a queue or fail immediately depends on your infrastructure and the nature of the task.

4.1 mysql2 vs mysql2/promise

Import	Behaviour
`mysql2`	Functions take **callbacks**
`mysql2/promise`	The same functions (same arguments) but they return **promises**; use async/await; no callback functions

It is always better to use **mysql2/promise**. Among its functions, **createConnection** creates a single connection (not application friendly); for enterprise development **createPool** is the only solution.

4.2 Creating the pool

```
import express from "express";
import { createPool } from "mysql2/promise";

const app = express();
app.use(express.json());          // register the JSON body middleware (from the morning session)

const sqlConnectionPool = createPool({
    host: "localhost",
    port: 3306,
    user: "root",
    password: "root",
    database: "productdb",
    connectionLimit: 10,
    queueLimit: 0
});
```

Option	Meaning
host	Address of the MySQL server. 'localhost' here; in real projects the IP or URL of the machine running MySQL
port	MySQL port: **3306** by default unless changed
user, password	MySQL user name (root) and its password
database	The database to connect to (productdb)
connectionLimit	Maximum connections in the pool (default 10)
queueLimit	Maximum number of connection requests the pool queues before returning an error. **0 means no limit** (unlimited queue). Choose a number if you want requests to be refused beyond that

createPool returns the pool object, which we keep in a reference variable (**sqlConnectionPool**). It is needed later for every operation: insert, select, update and delete.

> **Additional note (not from the lecture)**
> The default connectionLimit and the meaning of queueLimit: 0 are as in the mysql2 documentation (the instructor read the option description on screen). The password 'root' is only the instructor's local demo credential; never put real credentials directly in code, use environment variables.

5. Create: POST /product

Base route first (to check the server):

```
app.get("/", (request, response) => {
    response.send({ message: "Welcome to crud APIs" });
});
```

5.1 Error handling first

Instructor's explanation
Database connection failures or errors in the operation logic can crash the server. Such cases must be handled gracefully: wrap the logic in **try / catch**. If something goes wrong in the try block (including a rejected promise), control goes to the catch block, where you **log the error on the console for debugging** and send an appropriate response such as 'Something went wrong' with status **500**. If you do not handle it, the client gets no proper response when the server fails.

Status code: by default a response is 200; for an error it should be **500** (a generic server error). In real applications you put conditions on the error object to send different status codes and messages.

5.2 The three steps

1. **Get the data from the client** in the request body. The back-end developer decides the structure: keys *name*, *price*, *description*, *qty*. Read it with **request.body** (this works because express.json() was registered with app.use) and destructure it.
2. **Create the SQL INSERT query** (as a string).
3. **Run the query** in the database with the connection pool's **query** function (a promise, so use await and mark the handler async).

5.3 SQL injection and the ? placeholder

The first way one might write the query is by joining the values into the string:

```
// WRONG: concatenating data into the query (SQL injection risk)
const insertQry = `INSERT into product(name, price, description, qty)
                   VALUES('${name}', ${price}, '${description}', ${qty})`;
```

Common mistake
This is **100% prone to SQL injection attacks**: anyone can send malicious text in the fields and change the meaning of your query. **Never concatenate data into any query** (insert, select, update or delete). This is not specific to Express or MySQL2; it is true in every technology.

Instructor's explanation
Instead write a **question mark parameter** in place of each value, and pass the actual values separately in the **second argument (an array)** of the query function. The query with its question marks goes to the database first, and the values are filled in afterwards, in sequence: the first ? gets the first value (name), the second gets the price, the third the description, the fourth the quantity. This is a **prepared statement**. Other ways to avoid the problem are ORMs: in JavaScript, **Sequelize** and **Prisma** (a student also suggested prepared statements and ORMs, which is right; in Java, Spring Boot with Hibernate does the same job). The lecture goes with plain mysql2 from scratch.

```
app.post("/product", async (request, response) => {
    try {
        // step 1: get data from client {name:"", price: 6677, description:"", qty:8}
        const { name, price, description, qty } = request.body;

        // step 2: create a SQL INSERT query (values replaced by ? placeholders)
        const insertQry = `INSERT into product(name, price, description, qty) VALUES(?,?,?,?)`;

        // step 3: run this query in the db
        const [result] = await sqlConnectionPool.query(insertQry, [name, price, description, qty]);
        console.log(result.insertId);

        response.send({ message: "product inserted", insertedId: result.insertId });

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
```

```
});
```

Explanation: the id column is left out of the column list because it is auto increment. The order of the array [name, price, description, qty] must match the order of the question marks.

5.4 What the query function returns

Instructor's explanation
In the first test the instructor logged the whole result. It is an **array of objects**. The first element (index 0) is an object with information about the insert: **affectedRows** (1: one row was affected), **insertId** (the id generated for the new record) and some other fields. So the first object is read as `result[0].insertId`; to avoid indexing every time, **destructure the array**: `const [result] = await ...` takes the first element into *result*, and then `result.insertId` works. (The second element of the array holds field/column details.)

Testing in Postman: POST to http://127.0.0.1:7500/product with Body > raw > JSON:

```
{
    "name": "keyboard",
    "price": 5.4,
    "description": "wireless keyboard",
    "qty": 10
}

Output:
Response:       { "message": "product inserted", "insertedId": 1 }
Console:        1
(next requests: mouse -> insertedId 2; headphone -> insertedId 3)
```

Then SELECT * FROM product in MySQL shows the three rows (keyboard, mouse, headphone): the data is permanently stored in the database. This completes the **Create** operation. (The values in the request bodies are the instructor's demo values: keyboard 5.4, mouse 3.87, headphone 13.87 with quantity 16.)

6. Read: GET /product (all products)

The same URL **/product** can be used with a different request method. POST /product inserts a product; GET /product fetches all of them. The try/catch stays the same; the body is simpler because the client sends nothing:

```
app.get("/product", async (request, response) => {
    try {
        // step 1: create a select query
        const fetchAllProductsQry = "SELECT * FROM product";

        // step 2: run this query (no ? parameters, so no second argument)
        const [result] = await sqlConnectionPool.query(fetchAllProductsQry);

        response.send(result);

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});
```

Instructor's explanation
The result of the query function is always an array of two elements. The **first element is itself an array: the rows of the table converted into an array of JSON objects** (the table's rows and columns become JSON objects). The second element describes the table structure. So destructure the first element into *result* and send it directly; the response is exactly the array of JSON objects the front end needs.

```
Output:
```

```
GET /product  ->  [
  { "id": 1, "name": "keyboard", "price": 5.4,  "description": "wireless keyboard", "qty": 10 },
  { "id": 2, "name": "mouse",    "price": 3.87, "description": "wireless mouse",    "qty": ...},
  { "id": 3, "name": "headphone", ... }
]
```

Common mistake
In a real enterprise application you must **not** expose an API that returns all records. With a crore (10 million) of products the query takes long, the server may crash, the data transfer is huge and the browser becomes slow. The replacement is **pagination**: the user sees a limited number of records and selects a page number (1, 2, 3, ...) at the bottom of the table. The instructor asked you to explore pagination (paginated queries) as an assignment. Never send all the records in a single response.

Additional note (not from the lecture)
A hint (not given in the lecture): paginated queries usually use LIMIT and OFFSET with page number and page size sent as URL (query) parameters.

7. Read One: GET /product/:id

To fetch a product by its id, the id comes on the URL as a **URL parameter** (`:id`), read with **request.params** and destructured. The query uses a ? placeholder and the value is passed in an array:

```
app.get("/product/:id", async (request, response) => {
    try {
        // step 1: read id from url parameter
        const { id } = request.params;

        // step 2: create a select query
        const fetchByIdQuery = "SELECT * FROM product WHERE id = ?";

        // step 3: run this query
        const [result] = await sqlConnectionPool.query(fetchByIdQuery, [id]);

        if (result.length > 0) {
            response.send(result[0]);                // a single object, not an array
        } else {
            response.status(404).send({ message: `Product with id ${id} not found` });
        }

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});
```

Explanation: a SELECT always returns an array of rows, even for one row. Because id is the **primary key**, there can be at most one match, so we send **result[0]** (a single object). This matches what the front-end developer expects: if they ask for a single product, return a single product object, not an array. If the id does not exist, result is an empty array, so **result.length** is 0 and we send a **404** with a message.

```
Output:
GET /product/1  ->  200  { "id": 1, "name": "keyboard", "price": 5.4, "description": "wireless keyboard
GET /product/2  ->  200  { "id": 2, ... }
GET /product/4  ->  404  { "message": "Product with id 4 not found" }
```

Remember the status code: 404 means not found. (A browser may show cached responses sometimes; resend the request to see current data.)

8. Delete: DELETE /product/:id

The same URL again, with the **DELETE** method. Same three steps: read the id, create a DELETE query with a ? placeholder, run it:

```
app.delete("/product/:id", async (request, response) => {
    try {
        // step 1: read id from url parameter
        const { id } = request.params;

        // step 2: create a delete query
        const deleteQry = "DELETE FROM product WHERE id = ?";

        // step 3: run this query
        const [result] = await sqlConnectionPool.query(deleteQry, [id]);
        console.log(result);

        response.send({ message: "Product Deleted" });

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});

app.listen(7500);

Output:
DELETE /product/2   ->  { "message": "Product Deleted" }
Console result:     ... insertId: 0, affectedRows: 1 ...
GET /product/2      ->  404  { "message": "Product with id 2 not found" }
```

Instructor's explanation
affectedRows is 1 and insertId is 0 (nothing was inserted). The follow-up GET with id 2 returns 404, which proves the product was deleted. You do not have to use the primary key in your queries: you can customise them, for example by name (*WHERE name = ?*), a pattern (*WHERE name LIKE ?*), or price conditions. The primary key is just the common choice.

Same URL, different methods: /product/:id is used with GET to fetch and DELETE to delete. Only the request method changes.

9. Assignments

Assignment	What to do
Check existence before delete	If the product does not exist, send a **404** response instead of 'Product Deleted'. (Hint from the result: look at affectedRows.)
Update API	Method **PUT**, URL **/product/:id**. The id comes from the URL parameter and the new product data from the **request body**. Flow is the same as insert: read, build an UPDATE query with ? placeholders, run it. Also give a 404 if the product does not exist.
Duplicate (unique column) error	If a column has a UNIQUE constraint (for example a phone number or product name) and you insert a duplicate value, MySQL throws an error and the API would return 500. Handle it separately: the error object contains an **error code**; read it in the catch block with a condition, and send a clear message such as 'this phone number is already taken'.

Additional note (not from the lecture)
For the duplicate case, the mysql2 error object has `error.code === "ER_DUP_ENTRY"` (MySQL error number 1062). The instructor only said to read the error code; the exact code is my addition. A reasonable status for it is 409 (conflict), though the lecture did not state one.

10. The Complete index.js

```
import express from "express";
import { createPool } from "mysql2/promise";

const app = express();
app.use(express.json());

const sqlConnectionPool = createPool({
    host: "localhost",
    port: 3306,
    user: "root",
    password: "root",
    database: "productdb",
    connectionLimit: 10,
    queueLimit: 0
});

app.get("/", (request, response) => {
    response.send({ message: "Welcome to crud APIs" });
});

app.post("/product", async (request, response) => {
    try {
        const { name, price, description, qty } = request.body;
        const insertQry = `INSERT into product(name, price, description, qty) VALUES(?,?,?,?)`;
        const [result] = await sqlConnectionPool.query(insertQry, [name, price, description, qty]);
        console.log(result.insertId);
        response.send({ message: "product inserted", insertedId: result.insertId });
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});

app.get("/product", async (request, response) => {
    try {
        const fetchAllProductsQry = "SELECT * FROM product";
        const [result] = await sqlConnectionPool.query(fetchAllProductsQry);
        response.send(result);
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});

app.get("/product/:id", async (request, response) => {
    try {
        const { id } = request.params;
        const fetchByIdQuery = "SELECT * FROM product WHERE id = ?";
        const [result] = await sqlConnectionPool.query(fetchByIdQuery, [id]);
        if (result.length > 0) {
            response.send(result[0]);
        } else {
            response.status(404).send({ message: `Product with id ${id} not found` });
        }
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});

app.delete("/product/:id", async (request, response) => {
    try {
```

```
        const { id } = request.params;
        const deleteQry = "DELETE FROM product WHERE id = ?";
        const [result] = await sqlConnectionPool.query(deleteQry, [id]);
        console.log(result);
        response.send({ message: "Product Deleted" });
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});

app.listen(7500);
```

Route	Method	SQL	Success	Failure
/	GET	-	200 welcome message	-
/product	POST	INSERT	200, insertedId	500
/product	GET	SELECT *	200, array of products	500
/product/:id	GET	SELECT ... WHERE id = ?	200, one object	404 (not found), 500
/product/:id	DELETE	DELETE ... WHERE id = ?	200, Product Deleted	500 (404 check is your assignment)
/product/:id	PUT	UPDATE	assignment	assignment

11. Closing Remarks from the Session

- A question on **login, authentication and registration with a password**: to be covered later. The next topics are the front end: React (with TypeScript, ESLint) and **how to connect the front end with the back end**, done in React rather than plain HTML.
- A student asked whether the ORM route is better: yes, ORMs (Sequelize, Prisma in JavaScript) solve SQL injection out of the box, but learning an ORM takes time; this lecture uses mysql2 directly.
- Prefer parameterised queries every time; never concatenate.

12. Revision Material

12.1 Summary of all major topics

- The client calls the API; the API connects to the database; the client **never** accesses the database directly. Insert flow: request body, INSERT, response. Fetch flow: request, SELECT, rows converted to JSON array, response.
- **CRUD** = create, read, update, delete = INSERT, SELECT, UPDATE, DELETE = POST, GET, PUT, DELETE.
- **mysql2** is the third-party module to connect Node.js to MySQL; **mysql2/promise** returns promises (preferred).
- **Connection pool**: ready-made connections with a connection limit and a queue limit; connections are borrowed and released. Per-request connections do not scale.
- **createPool** options: host, port (3306), user, password, database, connectionLimit, queueLimit.
- Use **app.use(express.json())** to read JSON request bodies. Wrap logic in **try/catch**; log the error; send 500 'Something went wrong'.
- **Never concatenate** values in a SQL query (SQL injection). Use **? placeholders** and pass the values array to **pool.query(sql, values)** (prepared statement).
- **query()** returns an array: destructure **const [result]**. For an insert, result has insertId and affectedRows; for a select, result is an array of row objects; the second array element describes the fields.
- Single-record fetch: send **result[0]**; if **result.length** is 0 send **404**. Delete: **affectedRows** tells how many rows were deleted.
- Never expose 'get all' for big data; use **pagination**.

12.2 Important definitions and syntax

Term / Syntax	Meaning
CRUD	Create, Read, Update, Delete
Connection pool	Set of ready-made database connections shared by requests
Queue limit	How many requests may wait when all pooled connections are busy
SQL injection	Attack by injecting malicious SQL through concatenated input
Prepared statement / parameterised query	Query with ? placeholders; values supplied separately
`import { createPool } from "mysql2/promise";`	Import the pool creator (promise version)
`createPool({ host, port, user, password, database, connectionLimit, queueLimit })`	Create the pool
`const [result] = await pool.query(sql, [v1, v2]);`	Run a query safely and take the first element
`result.insertId`	The id generated by AUTO_INCREMENT
`result.affectedRows`	Number of rows changed or deleted
`response.status(404).send({...})`	Not found response
`response.status(500).send({...})`	Server error response
Pagination	Returning data page by page instead of everything

12.3 Key differences between related concepts

Concept A	Concept B	Key difference
Per-request connection	Connection pool	New connection each time (does not scale) vs reused ready-made connections
mysql2	mysql2/promise	Callbacks vs promises (async/await)
createConnection	createPool	One connection vs a pool of connections
Concatenated query	Parameterised query (?)	Vulnerable to SQL injection vs safe
GET /product	GET /product/:id	All rows (array) vs one row (object) or 404
POST /product	GET /product	Same URL, different method: insert vs fetch
queueLimit 0	queueLimit n	Unlimited waiting vs refuse beyond n
Fetch all	Pagination	Everything at once vs page by page

12.4 Common mistakes to avoid

Mistake	Why it fails	Correction
Concatenating name, price etc. into the SQL string	SQL injection	Use ? and the values array
Forgetting app.use(express.json())	request.body is undefined	Register the middleware before the routes
No try/catch around database calls	A failure crashes the request or server	Always catch, log, send 500
Putting the id in the INSERT column list	Auto-increment column	Leave it out
Values array in a different order than the ?	Wrong data in wrong columns	Keep the same sequence
Forgetting async/await on query	You get a promise, not the result	Use async handler and await
Sending result (array) for a single-product API	Front end expects one object	Send result[0]
Not handling a missing id	Empty array sent as success	Check result.length and send 404
Returning every record from a big table	Slow, crash, slow browser	Use pagination
Creating a new connection per request	Overloads the server	Use a connection pool
Using plain mysql2 with async/await	It uses callbacks	Import mysql2/promise

12.5 Interview questions and answers

Question	Answer
Why can the client not talk to the database directly?	Security and structure: the API (back end) creates the connection and runs controlled operations; the client only sends requests.
What is CRUD?	Create, Read, Update, Delete.
What is a connection pool and why use it?	A pool of ready-made database connections reused by requests; avoids creating a connection per request and limits load.
What are connectionLimit and queueLimit?	Maximum pooled connections, and the number of requests allowed to wait when all are busy (0 means unlimited).
What is mysql2/promise?	The promise-based API of the mysql2 module, used with async/await.
What is SQL injection and how do you prevent it?	Injecting SQL through input that is concatenated into a query; prevent with parameterised queries (? placeholders), prepared statements or an ORM.
What does pool.query return?	An array; the first element is the result (rows for SELECT, an info object with insertId and affectedRows for INSERT/UPDATE/DELETE).
How do you get the id of an inserted row?	result.insertId.
What status code for a missing product?	404.
What status code for an unexpected server error?	500.
Why avoid 'get all' APIs?	Large data makes the server slow or crash and overloads the browser; use pagination.
Can one URL serve several operations?	Yes, with different request methods (POST, GET, DELETE, PUT on /product).
What are ORMs?	Libraries (Sequelize, Prisma, Hibernate) mapping objects to tables; they avoid hand-written queries and help prevent injection.
(extra) How would you detect a duplicate key error?	Check error.code (ER_DUP_ENTRY in mysql2) in the catch block.

12.6 Practice exercises (solutions not included)

Easy

1. Create the productdb database and the product table and insert two rows manually in MySQL.
2. Explain in your own words why the client never connects to the database.
3. Draw the insertion and fetch flows as two diagrams.

Medium

1. Build the five routes and test each in Postman. Record the status codes you get.
2. Change the insert route to use string concatenation, then explain what a malicious name value could do. Revert to ?.
3. Add a 404 response to the delete route when the product does not exist.
4. Write GET /product/search that finds products whose name contains a given text (use LIKE with a placeholder).

Challenging

1. Write the update API (PUT /product/:id) with a request body and a 404 for a missing id.
2. Add pagination to GET /product using page and size values from the URL and LIMIT / OFFSET.

3. Add a UNIQUE constraint on name and handle the duplicate insert error with a clear message and a suitable status code.
4. Set queueLimit to a small number, send many simultaneous requests and describe what happens.

12.7 Final revision checklist

- [ ] I can explain the insertion and fetch flows between client, API and database.
- [ ] I can create a database and table with an auto-increment primary key.
- [ ] I can set up a project with express and mysql2 and type module.
- [ ] I can explain a connection pool, the connection limit and the queue limit.
- [ ] I know mysql2 vs mysql2/promise and use createPool.
- [ ] I can write the POST route with try/catch, destructuring and a ? parameterised insert.
- [ ] I can explain SQL injection and how placeholders prevent it.
- [ ] I know that query returns an array and I destructure the first element; I can read insertId and affectedRows.
- [ ] I can write GET all, GET by id (with a 404) and DELETE routes.
- [ ] I understand why fetch-all is bad and what pagination is.
- [ ] I can attempt the update API, a missing-id 404 and duplicate error handling.